SchoolCare Screening Priority Tool

1. Project Overview and Approved Scope

SchoolCare helps counsellors in Kenyan secondary schools decide which students to offer formal depression screening first. It uses non symptom psychosocial and demographic information to produce a priority score for human review. It does not diagnose depression, recommend treatment or replace a counsellor.

The approved proposal planned data preparation, comparison of three models, school grouped validation, a saved model, one student assessment, class ranking and CSV export. These core features are implemented in the current Streamlit prototype. Survey data was used for model development, while fictional student records can be used to demonstrate the application.

2. Planned Features and Current Status

Planned feature	Purpose	Status	Evidence
Data preparation	Prepare valid survey records	Completed	notebook/student_screening.ipynb; reports/step1_quality_report.csv
Three ML models	Compare prediction methods	Completed	Notebook; reports/step5_validation_summary.csv
School grouped evaluation	Test on schools not used for training	Completed	reports/school_split_summary.csv; step6_test_results.csv
Saved model	Use the trained model in the interface	Completed	dashboard/model_artifacts/selected_pipeline.joblib
One student assessment	Calculate one student priority score	Completed	dashboard/ui/one_student.py
Class ranking and export	Sort students and export results	Completed	dashboard/ui/rank_class.py
Age and gender evaluation	Examine differences between groups	Completed	reports/step6_age_results.csv; step6_gender_results.csv
Live school use	Evaluate with actual students	Outside scope	Research prototype; further safeguards required

Scope Decisions

The project uses Streamlit and a saved prediction pipeline without a separate API or database, as permitted by the approved proposal. The questionnaire collects approved non symptom inputs. Clinical use and collection of new identifiable student records were not part of the approved scope.

3. Architecture, Code Quality and Object-Oriented Programming

The interface receives student answers and sends them to the input validation and model components. The saved pipeline generates probabilities. The review component displays and exports ranked students. The interface does not assign prediction scores through fixed if statements.

Main Classes and Responsibilities

Class	Responsibility	File
ScreeningPriorityModel	Loads the saved trained model and calls predict_proba() to calculate scores.	dashboard/core/screening_priority_model.py
Preprocessor	Checks answer codes and prepares one student or a class for scoring.	dashboard/core/preprocessor.py
PredictionResult	Stores results consistently for display and export.	dashboard/core/prediction_result.py
ReviewSession	Manages records, repeated Student IDs and ranked results.	dashboard/core/review_session.py

Code Quality Practices

Each class has a clear responsibility. This supports encapsulation, abstraction, modularity and separation of concerns. Shared validation and result handling can be reused by different pages. The project separates interface code in dashboard/ui, core logic in dashboard/core, model files in dashboard/model_artifacts and tests in dashboard/tests. Inheritance and polymorphism were not introduced where they would add complexity without benefit.

How the Interface Uses the Trained Model

For an individual student or uploaded class, the Preprocessor checks the input data. ScreeningPriorityModel loads the saved pipeline using joblib.load(), places the 34 approved predictors in the expected order and calls predict_proba(). The fixed threshold selected during validation supports the review decision. ReviewSession manages records and ranks students by the calculated probabilities. A counsellor makes the final screening decision.

Technical Evidence

The repository contains the Streamlit application, reusable classes, saved pipeline, notebook, tests and evaluation output. The class definitions and method calls provide direct evidence of the object oriented design and trained model integration.

4. Algorithms and Technical Problem Solving

Data and Training

The project used Survey C of a Kenyan adolescent mental health dataset with 2,905 records. A total of 1,799 records from 32 schools had complete PHQ 8 responses and a school identifier. PHQ 8 was used only to create the training outcome. PHQ and GAD symptom inputs were excluded from prediction. The final pipeline uses 34 non symptom predictors.

Training and testing were separated by school. Cross validation also grouped students by school. The preprocessing pipeline handled missing data using information learned from training data, reducing leakage into validation and testing.

Model Comparison on Held Out Schools

Model	ROC AUC	Recall	Precision	Role
Logistic Regression	0.685	0.577	0.403	Selected
Random Forest	0.673	0.608	0.393	Comparison
Decision Tree	0.598	0.577	0.316	Comparison

Why Logistic Regression Was Selected

Logistic Regression was selected using the validation results before the final test. It offered a practical balance of ranking performance, simpler interpretation and stable use with the structured data. Random Forest achieved slightly better recall on the final test, but choosing a model after seeing those results would make the final evaluation less independent. The saved Logistic Regression model was therefore retained.

Results and Decision Threshold

The proposed targets were ROC AUC of at least 0.65 and recall of at least 0.60. The selected model met the ROC AUC target with 0.685, while its recall of 0.577 was below target. The threshold of 0.5034 was selected using validation data and fixed before final testing. Predicted probabilities are used to rank students for counsellor review; they are not diagnoses.

Group Results and Limitations

Recall on the final test was 0.735 for female students and 0.395 for male students. Recall was 0.714 for ages 13 to 15 and 0.540 for ages 16 to 21. These gaps require further study. Results from one historical dataset cannot establish safety or performance for every Kenyan school.

5. Testing, Error Handling and Product Evidence

Testing Approaches

Approach	What was checked	Recorded result
Automated unit and integration tests	Input validation, saved model scoring, threshold, callbacks, duplicate handling and review sessions.	37 tests passed with no failures in Python 3.11 with scikit learn 1.6.1. Total run: 2.096 seconds.
Manual functional testing	Student form, class CSV upload, ranking, exports and error messages in the Streamlit interface.	Local application started successfully. Screenshots of completed tasks and errors should be attached as evidence.

Test Cases and Error Handling

Duplicate Student ID: Student ID is the unique key used to identify each student record. When an ID is repeated, the system rejects duplicate IDs in a class upload or asks for confirmation before replacing an existing individual assessment. This prevents the same student from appearing as multiple records or being replaced by mistake.

Missing Student ID: Each assessment needs a Student ID to identify the student whose score is being reviewed. A missing or blank ID is rejected instead of creating an unidentified result. This is a required manual test case; the automated test evidence confirms duplicate ID handling, but not a separate passing missing ID test.

Other automated checks cover invalid answer codes, missing questionnaire answers, probability values between zero and one, consistent results when column order changes and correct use of the saved pipeline and locked threshold.

Performance and Compatibility

The 37 automated tests passed in the project virtual environment on Windows using Python 3.11 and scikit learn 1.6.1. The full suite took 2.096 seconds. This is the test run duration, not class prediction time. Comparison across browsers, devices and different class sizes must be recorded separately if completed. No unverified speed or device results are claimed.

Improvements and Next Steps

Validation, duplicate record handling, saved model scoring and structured review sessions support reliable operation. Future work includes broader school evaluation, subgroup performance review, access control, privacy protection and feedback from counsellors before actual school use.

Repository and Demonstration Evidence

GitHub repository: Github repo

Deployed application: Deployed SchoolCare App

Walkthrough video: Demo Video